

Architecture des processeurs et optimisation

November 2016

Documents allowed - Duration 2h

An unjustified answer will be considered as false

It is imperative to use the provided squared sheets for the simplified diagrams

We would like to compare two implementation of the Mips-32 architecture.

The first, called **Mips**, is the 5-stage implementation (**IFC**, **DEC**, **EXE**, **MEM** and **WBK**) studied in class.

The second, called **MipsO**, is also a 5-stage implementation : **IFC**, **DEC**, **EXE**, **MEM** and **WBK**. The **MipsO** stages are identical to those of Mips.

The aim of this comparison is to study the impact of implementing bypasses involving *load* instructions. In **Mips**, when a *load* instruction is followed by an instruction that depends on the result of the *load*, bypasses are used to avoid stall cycles. These bypasses catch the memory operand read by the *load* instruction in its **MEM** stage and make it available to the instruction that needs the operand.

We would like to study the impact of eliminating these bypasses in the **MipsO** implementation.

- 1- Without considering the optimization steps of the compiler, is a compiler for **MipsO** different from a compiler for **Mips** ?
- 2- In an even more simplified implementation of the processor, data dependencies involving the result of *load* instructions are not handled by hardware, neither by bypasses nor by the generation of stall cycles.
Does this simplification have any impact on the compiler ?
- 3- In **MipsO**, by eliminating bypasses on memory operands, we hope to increase the processor's frequency. Explain why we can expect such an improvement.
- 4- In **Mips** there are 8 bypasses to limit performance degradation due to data dependencies. What are the bypasses in the **MipsO** implementation ? Illustrate your answer using a simplified diagrams. For each bypass, give an example of an instruction sequence that illustrates the need for this bypass.

We'd like to compare these 2 implementations on a 'grayscale' image processing program.

An image is made up of a set of pixels organized as an array. For a grayscale image, each pixel is coded on a byte. The value 0 represents the color black, the value 255 the color white. Values between 0 and 255 represent the different shades of gray.

Often, image processing functions are not applied to the whole image, but only to a part of it. For this purpose, a mask is used. For each pixel, the mask indicates whether the function is applied to pixel or not. If the mask is 0, the pixel is not modified. If the mask value is 255, the processing function is applied. The image mask is therefore an array of bytes of the same size as the image.

The following function allows to obtain the negative image on a portion of the image.

```

unsigned char *neg_msk (unsigned char *pt_img,
                        unsigned char *pt_msk,
                        unsigned int   size )
{
    unsigned int i = 0 ;
    for (i=0 ; i!=size ; i++)
    {
        if (pt_msk [i] != 0)
            pt_img [i] = 255 - pt_img [i];
    }
    return (pt_img);
}

```

A non-pipelined compiler has produced the following assembly code for the main loop of this function. It is assumed that, at the loop entry, register R4 contains the address of array **pt_img**, R5 the address of array **pt_msk** and register R6 the end address of array **pt_img** (the address immediately following the last element of the array). Register R7 contains the value 255.

Version 1 :

```

    _For : Lbu      r7 , 0 (r5 )      ; pt_msk[i]
           Bne      r7 , r0 , _Endif
           Lbu      r8 , 0 (r4 )      ; pt_img[i]
           Sub      r8 , r7 , r8      ; 255-pt_img[i]
           Sb       r8 , 0 (r4 )
    _Endif: Addiu   r5 , r5 , 1
           Addiu   r4 , r4 , 1
           Bne      r4 , r6 , _For

```

- 5- Modify the code so it can be executed on **Mips** then, analyze the execution of the loop using a simplified diagram in the case the branch fails (show the bypasses on the diagram).

How many cycles are required to execute one iteration of the loop in the case of a failed branch and in the case of a successful branch ? Calculate the CPI and CPI-useful, assuming that in 25% of cases the branch fails.

- 6- Modify the code so it can be executed on **MipsO** then, analyze the execution using a simplified diagram in the case of a failed branch (show the bypasses on the diagram).

How many cycles are required to execute one iteration of the loop in the case of a failed branch and in the case of a successful branch? Calculate the CPI and CPI-useful, assuming that in 25% of cases the branch fails.

- 7- Optimize the code for **MipsO** by modifying the order of instructions to avoid lost cycles as much as possible (explain your approach). How many cycles are required to perform an iteration on **MipsO** ? You don't need to draw a diagram, but simply indicate the lost cycles on the code after optimization.

Same question for **Mips** implementation.

- 8- To improve the execution performance, we decide to unroll the loop code once (processing 2 elements at each iteration). Propose an optimized code for **MipsO**. How many cycles are required on **MipsO** to process an element ? You don't need to draw a diagram, but simply indicate the lost cycles on the code after optimization.

Same question for **Mips**.

Another (non-pipeline) compiler has produced a different version for the loop code, noting that when calculating on 8 bits : $255 - x = \bar{x}$

Version 2 :

```

_For :  Lbu      r7 , 0 (r5 )      ; pt_msk[i]
        Lbu      r8 , 0 (r4 )      ; pt_img[i]
        Xor      r8 , r7 , r8
        Sb       r8 , 0 (r4 )

        Addiu     r5 , r5 , 1
        Addiu     r4 , r4 , 1
        Bne      r4 , r6 , _For

```

- 9- Modify the code of Version 2 to adapt it to **MipsO**. How many cycles are needed to execute one iteration of the loop ? Calculate the CPI-useful. It is not necessary to draw a diagram, but simply indicate the lost cycles on the code.

Same question for **Mips**.

- 10- Optimize the code by modifying the order of instructions for **MipsO** to avoid lost cycles as much as possible. How many cycles are needed to perform one iteration ? You don't need to draw a diagram, but simply show the lost cycles on the code after optimization.

Same question for **Mips**.

- 11- To improve execution performance, we decide to unroll the code once (processing 2 elements at each iteration). Propose an optimized code for **MipsO**. How many cycles are needed to process an element ? You don't need to draw a diagram, but simply indicate the lost cycles on the code after optimization.

Same question for **Mips**.

- 12- For each of the 6 versions of the code :

- ☐ Version 1 : original code, reordered code, unrolled code
- ☐ Version 2 : original code, reordered code, unrolled code

what gain in frequency must be achieved for the **MipsO** to outperform the **Mips** realization ?